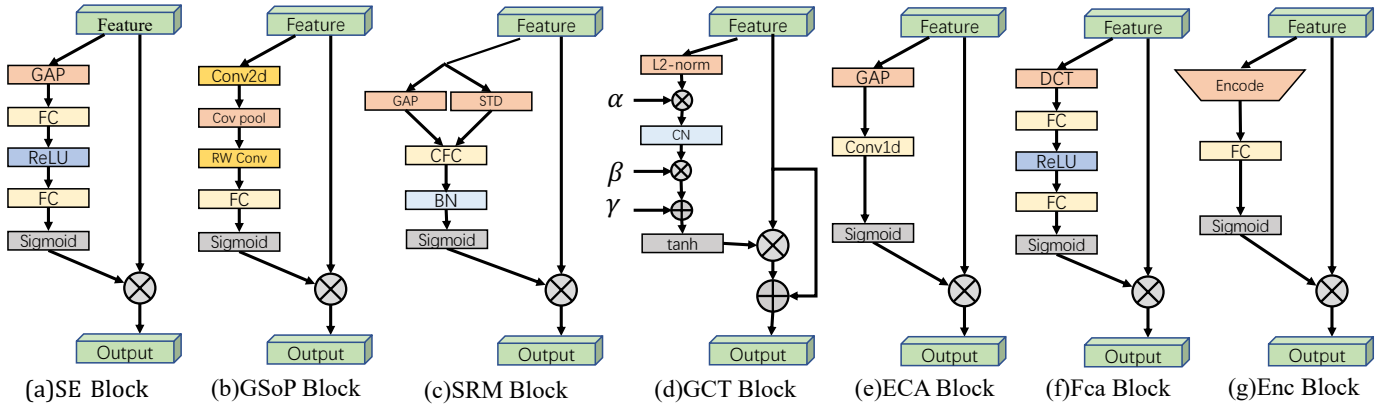

Fig. 5. Various channel attention mechanisms. GAP=global average pooling, GMP=global max pooling, FC=fully-connected layer, Cov pool=Covariance pooling, RW Conv=row-wise convolution, CFC=channel-wise fully connected, CN=channel normalization, DCT=discrete cosine transform.

TABLE 3
Representative channel attention mechanisms ordered by category and publication date. Their key aims are to emphasize important channels and capture global information. Application areas include: Cls = classification, Det = detection, SSeg = semantic segmentation, ISeg = instance segmentation, ST = style transfer, Action = action recognition.$g(x)$ and $f(g(x), x)$ are the attention process described by Eq. 1. Ranges means the ranges of attention map. S or H means soft or hard attention.(A) channel-wise product. (I) emphasize important channels, (II) capture global information.

Category	Method	Publication	Tasks	$g(x)$	$f(g(x), x)$	Ranges	S or H	Goals
Squeeze-and-excitation network	SENet [5]	CVPR2018	Cls, Det	global average pooling -> MLP -> sigmoid.	(A)	(0,1)	S	(I),(II)
Improve squeeze module	EncNet [53]	CVPR2018	SSeg	encoder -> MLP -> sigmoid.	(A)	(0,1)	S	(I),(II)
	GSoP-Net [54]	CVPR2019	Cls	2nd-order pooling -> convolution & MLP -> sigmoid	(A)	(0,1)	S	(I),(II)
	FcaNet [57]	ICCV2021	Cls, Det, ISeg	discrete cosine transform -> MLP -> sigmoid.	(A)	(0,1)	S	(I),(II)
Improve excitation module	ECANet [37]	CVPR2020	Cls, Det, ISeg	global average pooling -> conv1d -> sigmoid.	(A)	(0,1)	S	(I),(II)
Improve both squeeze and excitation module	SRM [55]	arXiv2019	Cls, ST	style pooling -> convolution & MLP -> sigmoid.	(A)	(0,1)	S	(I),(II)
	GCT [56]	CVPR2020	Cls, Det, Action	compute $L2$-norm on spatial -> channel normalization -> tanh.	(A)	(-1,1)	S	(I),(II)

given the input feature map X, bi-attention first uses bilinear pooling to capture second-order information

$$\widetilde{x} = \mathrm{Bi}(\phi(X)) = \mathrm{Vec}(\mathrm{UTri}(\phi(X)\phi(X)^T)) \tag{25}$$

where ϕ denotes an embedding function used for dimensionality reduction, $\phi(x)^T$ is the transpose of $\phi(x)$ across the channel domain, $\mathrm{Utri}(\cdot)$ extracts the upper triangular elements of a matrix and $\mathrm{Vec}(\cdot)$ is vectorization. Then bi-attention applies the inner channel attention mechanism to the feature map $\widetilde{x} \in \mathbb{R}^{\frac{c'(c'+1)}{2} \times H \times W}$

$$\widehat{x} = \omega(\mathrm{GAP}(\widetilde{x}))\varphi(\widetilde{x}) \tag{26}$$

Here ω and φ are embedding functions. Finally the output feature map $\widehat{x}$ is used to compute the spatial channel attention weights of the outer point-wise attention mechanism:

$$s = \sigma(\widehat{x}) \tag{27}$$

$$Y = sX \tag{28}$$

The bi-attention block uses bilinear pooling to model the local pairwise feature interactions along each channel, while preserving the spatial information. Using the proposed AiA, the model pays more attention to higher-order statistical information compared with other attention-based models. Bi-attention can be incorporated into any CNN backbone to improve its representational power while suppressing noise.

3.3 Spatial Attention

Spatial attention can be seen as an adaptive spatial region selection mechanism: *where to pay attention*. As Fig. 4 shows, RAM [31], STN [32], GENet [61] and Non-Local [15] are representative of different kinds of spatial attention methods. RAM represents RNN-based methods. STN represents those use a sub-network to explicitly predict relevant regions. GENet represents those that use a sub-network implicitly to predict a soft mask to select important regions. Non-Local represents self-attention related methods. In this subsection,

we first summarize representative spatial attention mechanisms and specify process $g(x)$ and $f(g(x), x)$ described as Eq. 1 in Tab. 4, then discuss them according to Fig. 4.

3.3.1 RAM

Convolutional neural networks have huge computational costs, especially for large inputs. In order to concentrate limited computing resources on important regions, Mnih et al. [31] proposed the *recurrent attention model* (RAM) that adopts RNNs [147] and reinforcement learning (RL) [148] to make the network learn where to pay attention. RAM pioneered the use of RNNs for visual attention, and was followed by many other RNN-based methods [21], [35], [88].

As shown in Fig. 6, the RAM has three key elements: (A) a glimpse sensor, (B) a glimpse network and (C) an RNN model. The glimpse sensor takes a coordinate l_{t-1} and an image X_t. It outputs multiple resolution patches $\rho(X_t, l_{t-1})$ centered on l_{t-1}. The glimpse network $f_g(\theta(g))$ includes a glimpse sensor and outputs the feature representation g_t for input coordinate l_{t-1} and image X_t. The RNN model considers g_t and an internal state h_{t-1} and outputs the next center coordinate l_t and the action a_t, e.g. the softmax result in an image classification task. Since the whole process is not differentiable, it applies reinforcement learning strategies in the update process.

This provides a simple but effective method to focus the network on key regions, thus reducing the number of calculations performed by the network, especially for large inputs, while improving image classification results.

3.3.2 Glimpse Network

Inspired by how humans perform visual recognition sequentially, Ba et al. [88] proposed a deep recurrent network, similar to RAM [31], capable of processing a multi-resolution crop of the input image, called a glimpse, for multiple object recognition task. The proposed network updates its hidden state using a glimpse as input, and then predicts a new object as well as the next glimpse location at each step. The glimpse is usually much smaller than the whole image, which makes the network computationally efficient.

The proposed deep recurrent visual attention model consists of a context network, glimpse network, recurrent network, emission network, and classification network. First, the context network takes the down-sampled whole image as input to provide the initial state for the recurrent network as well as the location of the first glimpse. Then, at the current time step t, given the current glimpse x_t and its location tuple l_t, the goal of the glimpse network is to extract useful information, expressed as

$$g_t = f_{\text{image}}(X) \cdot f_{\text{loc}}(l_t) \tag{29}$$

where $f_{\text{image}}(X)$ and $f_{\text{loc}}(l_t)$ are non-linear functions which both output vectors having the same dimension, and $\cdot$ denotes element-wise product, used for fusing information from two branches. Then, the recurrent network, which consists of two stacked recurrent layers, aggregates information gathered from each individual glimpse. The outputs of the recurrent layers are:

$$r_t^{(1)} = f_{\text{rec}}^{(1)}(g_t, r_{t-1}^{(1)}) \tag{30}$$

$$r_t^{(2)} = f_{\text{rec}}^{(2)}(r_t^{(1)}, r_{t-1}^{(2)}) \tag{31}$$

Given the current hidden state $r_t^{(2)}$ of the recurrent network, the emission network predicts where to crop the next glimpse. Formally, it can be written as

$$l_{t+1} = f_{\text{emis}}(r_t^{(2)}) \tag{32}$$

Finally, the classification network outputs a prediction for the class label y based on the hidden state $r_t^{(1)}$ of the recurrent network

$$y = f_{\text{cls}}(r_t^{(1)}) \tag{33}$$

Compared to a CNN operating on the entire image, the computational cost of the proposed model is much lower, and it can naturally tackle images of different sizes because it only processes a glimpse in each step. Robustness is additionally improved by the recurrent attention mechanism, which also alleviates the problem of over-fitting. This pipeline can be incorporated into any state-of-the-art CNN backbones or RNN units.

3.3.3 Hard and soft attention

To visualize where and what an image caption generation model should focus on, Xu et al. [35] introduced an attention-based model as well as two variant attention mechanisms, *hard attention* and *soft attention*.

Given a set of feature vectors $\boldsymbol{a} = \{a_1, \ldots, a_L\}, a_i \in \mathbb{R}^D$ extracted from the input image, the model aims to produce a caption by generating one word at each time step. Thus they adopt a long short-term memory (LSTM) network as a decoder; an attention mechanism is used to generate a contextual vector z_t conditioned on the feature set $\boldsymbol{a}$ and the previous hidden state h_{t-1}, where t denotes the time step. Formally, the weight $\alpha_{t,i}$ of the feature vector a_i at the t-th time step is defined as

$$e_{t,i} = f_{\text{att}}(a_i, h_{t-1}) \tag{34}$$

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{k=1}^{L} \exp(e_{t,k})} \tag{35}$$

where f_{att} is implemented by a multilayer perceptron conditioned on the previous hidden state h_{t-1}. The positive weight $\alpha_{t,i}$ can be interpreted either as the probability that location i is the right place to focus on (hard attention), or as the relative importance of location i to the next word (soft attention). To obtain the contextual vector z_t, the hard attention mechanism assigns a multinoulli distribution parametrized by $\{\alpha_{t,i}\}$ and views z_t as a random variable:

$$p(s_{t,i} = 1 | \boldsymbol{a}, h_{t-1}) = \alpha_{t,i} \tag{36}$$

$$z_t = \sum_{i=1}^{L} s_{t,i} a_i \tag{37}$$

On the other hand, the soft attention mechanism directly uses the expectation of the context vector z_t,

$$z_t = \sum_{i=1}^{L} \alpha_{t,i} a_i \tag{38}$$

The use of the attention mechanism improves the interpretability of the image caption generation process by allowing the user to understand what and where the model is focusing on. It also helps to improve the representational capability of the network.

TABLE 4
Representative spatial attention mechanisms sorted by category and date. Application areas include: Cls = classification, FGCls = fine-grained classification, Det = detection, SSeg = semantic segmentation, ISeg = instance segmentation, ST = style transfer, Action = action recognition, ICap = image captioning.$g(x)$ and $f(g(x), x)$ are the attention process described by Eq. 1. Ranges means the ranges of attention map. S or H means soft or hard attention.(A) choose region according to the prediction. (B) element-wise product, (C)aggregate information via attention map. (I)focus the network on discriminative regions, (II) avoid excessive computation for large input images, (III) provide more transformation invariance, (IV) capture long-range dependencies, (V) denoise input feature map (VI) adaptively aggregate neighborhood information, (VII) reduce inductive bias.

Category	Method	Publication	Tasks	$g(x)$	$f(g(x), x)$	Ranges	S or H	Goals
RNN-based methods	RAM [31]	NIPS2014	Cls	use RNN to recurrently predict important regions	(A)	(0,1)	H	(I), (II).
	Hard and soft attention [35]	ICML2015	ICap	a)compute similarity between visual features and previous hidden state -> interpret attention weight.	(C)	(0,1)	S, H	(I).
Predict the relevant region explictly	STN [32]	NIPS2015	Cls, FGCls	use sub-network to predict an affine transformation.	(A)	(0,1)	H	(I), (III).
	DCN [7]	ICCV2017	Det, SSeg	use sub-network to predict offset coordinates.	(A)	(0,1)	H	(I), (III).
Predict the relevant region implictly	GENet [61]	NIPS2018	Cls, Det	average pooling or depth-wise convolution -> interpolation -> sigmoid	(B)	(0,1)	S	(I).
	PSANet [87]	ECCV2018	SSeg	predict an attention map using a sub-network.	(C)	(0,1)	S	(I), (IV).
Self-attention based methods	Non-Local [15]	CVPR2018	Action, Det, ISeg	Dot product between query and key -> softmax	(C)	(0,1)	S	(I), (IV), (V)
	SASA [43]	NeurIPS2019	Cls, Det	Dot product between query and key -> softmax.	(C)	(0,1)	S	(I), (VI)
	ViT [34]	ICLR2021	Cls	divide the feature map into multiple groups -> Dot product between query and key -> softmax.	(C)	(0,1)	S	(I),(IV), (VII).

3.3.4 Attention Gate

Previous approaches to MR segmentation usually operate on particular regions of interest (ROI), which requires excessive and wasteful use of computational resources and model parameters. To address this issue, Oktay et al. [19] proposed a simple and yet effective mechanism, the *attention gate* (AG), to focus on targeted regions while suppressing feature activations in irrelevant regions.

Given the input feature map X and the gating signal $G \in \mathbb{R}^{C' \times H \times W}$ which is collected at a coarse scale and contains contextual information, the attention gate uses additive attention to obtain the gating coefficient. Both the input X and the gating signal are first linearly mapped to an $\mathbb{R}^{F \times H \times W}$ dimensional space, and then the output is squeezed in the channel domain to produce a spatial attention weight map $S \in \mathbb{R}^{1 \times H \times W}$. The overall process can be written as

$$S = \sigma(\varphi(\delta(\phi_x(X) + \phi_g(G)))) \tag{39}$$

$$Y = SX \tag{40}$$

where φ, ϕ_x and ϕ_g are linear transformations implemented as 1×1 convolutions.

The attention gate guides the model's attention to important regions while suppressing feature activation in unrelated areas. It substantially enhances the representational power of the model without a significant increase in computing cost or number of model parameters due to its lightweight design. It is general and modular, making it simple to use in various CNN models.

3.3.5 STN

The property of translation equivariance makes CNNs suitable for processing image data. However, CNNs lack other transformation invariance such as rotational invariance, scaling invariance and warping invariance. To achieve these attributes while making CNNs focus on important regions, Jaderberg et al. [32] proposed *spatial transformer networks* (STN) that use an explicit procedure to learn invariance to translation, scaling, rotation and other more general warps, making the network pay attention to the most relevant regions. STN was the first attention mechanism to explicitly predict important regions and provide a deep neural network with transformation invariance. Various following works [7], [36] have had even greater success.

Taking a 2D image as an example, a 2D affine transformation can be formulated as:

$$\begin{bmatrix} \theta_{11} & \theta_{12} & \theta_{13} \\ \theta_{21} & \theta_{22} & \theta_{23} \end{bmatrix} = f_{\text{loc}}(U) \tag{41}$$

$$\begin{pmatrix} x_i^s \\ y_i^s \end{pmatrix} = \begin{bmatrix} \theta_{11} & \theta_{12} & \theta_{13} \\ \theta_{21} & \theta_{22} & \theta_{23} \end{bmatrix} \begin{pmatrix} x_i^t \\ y_i^t \\ 1 \end{pmatrix}. \tag{42}$$

Here, U is the input feature map, and f_{loc} can be any differentiable function, such as a lightweight fully-connected network or convolutional neural network. x_i^s and y_i^s are